

Informe Proyecto MLOps

Sistema de Predicción de Default de Clientes

Integrantes: Álvaro Gómez, Camilo Silva, Germán Ruiz

Fecha: 16/08/2026

1. Problema

El proyecto aborda el problema de predicción del riesgo de incumplimiento de clientes utilizando información relacionada con sus características personales, laborales, ingresos y nivel de endeudamiento.

El objetivo es desarrollar una solución completa de Machine Learning que no se limite al entrenamiento de un modelo, sino que permita:

- Entrenar el modelo de manera reproducible;
- Evaluar su desempeño sobre datos no vistos;
- Almacenar el modelo y sus metadatos;
- Exponer predicciones mediante una API REST;
- Ejecutar la solución mediante Docker;
- Automatizar pruebas y validaciones mediante GitHub Actions;
- Facilitar la reproducción del sistema por parte de otros integrantes del equipo.

La variable objetivo es del tipo binaria (0 o 1), que predice la probabilidad de cumplimiento de pago de un crédito de clientes (1) o no (0), mientras que las variables predictoras corresponden a edad, nivel educacional, años trabajando, ingresos y diferentes tipos de deuda.

- Datos y preparación

El dataset utilizado contiene información histórica de clientes.

Las variables utilizadas por el modelo son:

Variable	Tipo
Edad	Integer
Nivel_Educacional	Catégorica
Años_Trabajando	Integer
Ingresos	Integer
Deuda_Comercial	Numérica
Deuda_Credito	Numérica
Otras_Deudas	Numérica

Categorías para “Nivel_Educacional”

Categoría	Definición
SupInc	Superior Incompleta
Med	Media
Bas	Básica
Posg	Posgrado
SupCom	Superior Completa

El dataset original contiene 12.356 registros. Durante la preparación se eliminan duplicados considerando las variables utilizadas como entrada del modelo, obteniéndose finalmente 1.500 registros.

La separación entre entrenamiento y prueba se realiza mediante una división estratificada con 70% para entrenamiento y 30% para prueba. Se definió una semilla fija `random_state = 0` que permite reproducir la misma separación de datos y facilita la comparación entre ejecuciones.

2. Modelo y decisiones de diseño

Se seleccionó HistGradientBoostingClassifier como algoritmo de clasificación, luego el procesamiento de los datos se implementó mediante un Pipeline de scikit-learn, integrando el preprocesamiento y el modelo en un único artefacto. Posteriormente, para las variables numéricas se utiliza imputación mediante la mediana, mientras que para Nivel_Educacional se utiliza imputación por la categoría más frecuente y posteriormente codificación One-Hot.

La principal decisión de diseño fue serializar el pipeline completo, en lugar de guardar solamente el clasificador. De esta manera, el mismo procesamiento utilizado durante el entrenamiento se aplica posteriormente cuando la API recibe una nueva solicitud.

El artefacto generado es: models/model.joblib, el cual guarda el modelo serializado para que la API pueda desplegarlo.

Además, se genera: models/metadata.json que contiene información sobre el modelo utilizado y métricas evaluadas el quality gate. Esto es versión Python y scikit-learn. También fecha de train, la semilla, %train/test, features, tipos de variables y métricas del modelo.

El modelo se expone mediante una API REST implementada con **FastAPI** que contempla los siguientes endpoints:

- GET /health: Verifica el estado del servicio y que el modelo haya sido cargado correctamente.
- POST /predict: Devuelve la predicción y probabilidad estimada de cumplimiento
- POST /predict/batch: Permite realizar predicciones para múltiples clientes.
- GET /model/schema: Entrega las features esperadas, sus tipos y clasificación entre variables numéricas y categóricas.

Las entradas y salidas se validan mediante modelos Pydantic. En caso de recibir información inválida, la API devuelve error HTTP 422 y 200 para el caso de datos válidos.

Para la contenerización, se ejecuta mediante Docker utilizando una imagen base python:3.12-slim y las dependencias se encuentran fijadas por versión en requirements.txt.

El proyecto incluye docker-compose.yml, que levanta la API mediante “docker compose up –build”, esto permite que otra persona pueda clonar el repositorio y ejecutar la aplicación sin tener que configurar manualmente el entorno de ejecución.

En cuanto a calidad y test automatizados, se usa pytest, que cubre el estado del servicio, carga del modelo, contrato de los endpoints, predicciones individuales, predicciones batch, validación de campos obligatorios, valores fuera de rango, tipos de datos incorrectos, valores negativos en variables que no los permiten, casos borde, quality gate de métricas.

Actualmente las pruebas se ejecutan sin necesidad de conexión a servicios externos ni credenciales.

Además, se utiliza Ruff para linting y formato del código. Los comandos utilizados son “pytest -q”, “ruff check .” y “ruff format --check .”

Para integración y despliegue continuo (CI/CD) el repositorio utiliza GitHub Actions para automatizar la validación del proyecto.

El pipeline contempla una secuencia de etapas:



La prueba de humo levanta el contenedor y comprueba el funcionamiento real de la API mediante:

- GET /health
- POST /predict

De esta forma, no solamente se verifica que el código pueda ser ejecutado, sino que también se comprueba que la aplicación contenerizada responda correctamente.

El proyecto también contempla la publicación de la imagen Docker en GitHub Container Registry (GHCR) como parte del proceso de entrega.

Respecto a reproducibilidad y para permitir que cualquiera pueda reproducir el sistema, se incluyen requirements.txt con versiones fijadas, Dockerfile, docker-compose.yml, modelo serializado, metadata del modelo, configuración mediante .env.example, pruebas automatizadas y workflows de GitHub Actions.

La reproducción requiere ejecutar “docker compose up –build” y se puede comprobar el servicio mediante “curl.exe http://localhost:8000/health”

3. Resultados

La evaluación se realiza sobre el conjunto de prueba, que no participa en el entrenamiento.

Los resultados obtenidos fueron:

Métrica	Resultado
Accuracy	0.72
F1	0.79
ROC-AUC	0.78

El proyecto incorpora un quality gate que verifica los valores mínimos de 0,75 para F1 y ROC-AUC antes de considerar válido el modelo para las siguientes etapas del pipeline. El modelo obtenido supera ambos umbrales.

Al momento de correr la aplicación se obtuvieron resultados esperados. Como se puede mostrar en el anexo I, con alto nivel educacional e ingresos y baja deuda, la probabilidad de cumplimiento es alta (Prácticamente 1), mientras que en el Anexo II, se obtiene una baja probabilidad de cumplimiento con nivel de educación básico, bajo ingreso y alta deuda.

En el Anexo III, se observa que para valores fuera de rango (17 o 101 años), el modelo entrega error 422, mientras que en casos bordes (18 años), la predicción es válida.

Finalmente, en el anexo IV se observa que la integración y despliegue continuo se ejecuta correctamente.

4. Limitaciones, distribución del trabajo y conclusiones

El conjunto de datos disponible es relativamente limitado después de eliminar duplicados, pasando de 12.356 registros a 1.500 observaciones.

Además, el modelo se evalúa mediante una única separación train/test. Una mejora futura sería utilizar validación cruzada para obtener una estimación más robusta del desempeño.

El proyecto tampoco incorpora actualmente monitoreo de data drift o model drift, por lo que no existe una alerta automática ante cambios en la distribución de los datos de producción.

Otra limitación corresponde a la ausencia de un sistema especializado de tracking de experimentos y registro de versiones de modelos, como MLflow.

Finalmente, la API está diseñada como una app sencilla para fines académicos y no incorpora mecanismos avanzados de autenticación, escalamiento horizontal o infraestructura cloud.

Para trabajo futuro, con mayor tiempo, se podría trabajar en:

- MLflow para tracking de experimentos y versionado de modelos.
- Monitoreo de data drift y model drift.
- Reentrenamiento automático ante degradación del modelo.
- Validación cruzada durante el entrenamiento.
- Mayor cantidad y diversidad de datos.
- Despliegue en infraestructura cloud.
- Autenticación y autorización de la API.
- Métricas y monitoreo de la API en producción.
- Estrategias de escalamiento.
- Automatización del ciclo completo de entrenamiento y despliegue.

Distribución del trabajo

Integrante	Trabajo realizado
Álvaro Gómez	Diseño e implementación inicial del proyecto, entrenamiento del modelo, pipeline de Machine Learning, API FastAPI, configuración del predictor y contenerización Docker. Colaboración en informe
Germán Ruiz	Desarrollo y ampliación de pruebas automatizadas, validación de entradas, casos borde, pruebas de endpoints y quality gate. Colaboración y subida de informe.
Camilo Silva	Diseño del modelo de Machine Learning, Revisión y mejoras de CI/CD, pruebas de smoke test, documentación del proyecto y README. Colaboración en informe

Conclusión

El proyecto implementa un flujo MLOps completo para un modelo de predicción. Además, integra entrenamiento reproducible, evaluación sobre datos no vistos, serialización del modelo y sus metadatos, API REST, validación de entradas, pruebas automatizadas, contenerización y automatización mediante GitHub Actions.

La arquitectura permite que el sistema sea ejecutado de manera reproducible por los integrantes del equipo a partir del repositorio, reduciendo la dependencia de configuraciones específicas de cada computador.

Como siguiente etapa, el proyecto podría evolucionar hacia un entorno productivo incorporando monitoreo, versionado formal de modelos, tracking de experimentos y despliegue en infraestructura cloud.

Anexos

Anexo I: Predicción de cumplimiento de pago 1

Default Prediction API 1.0.0 OAS 3.1

openapi.json

API para predicción de riesgo de Default de clientes.

default

GET /health Health

POST /predict Predict

Realiza una predicción para un cliente.

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "Edad": 30,
  "Nivel_Educacional": "Posg",
  "Años_Trabajando": 15,
  "Ingresos": 1000,
  "Deuda_Comercial": 2,
  "Deuda_Credito": 5.5,
  "Otras_Deudas": 0
}
```

Curl

```
curl -X 'POST' \
  'http://localhost:8080/predict' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "Edad": 30,
    "Nivel_Educacional": "Posg",
    "Años_Trabajando": 15,
    "Ingresos": 1000,
    "Deuda_Comercial": 2,
    "Deuda_Credito": 5.5,
    "Otras_Deudas": 0
  }'
```

Request URL

http://localhost:8080/predict

Server response

Code Details

200

Response body

```
{
  "prediction": 1,
  "probability": 0.9979504986285883
}
```

Response headers

```
content-length: 49
content-type: application/json
date: Sat, 15 Aug 2020 15:06:13 GMT
server: uvicorn
```

Code Description Links

200 Successful Response No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "prediction": 0,
  "probability": 1
}
```

422 Validation Error No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

POST /predict/batch Predict Batch

GET /model/schema Model Schema

Anexo II: Predicción de cumplimiento de pago 2

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Edad": 65,
    "Nivel_Educacional": "Bas",
    "Años_Trabajando": 4,
    "Ingresos": 10,
    "Deuda_Comercial": 15,
    "Deuda_Credito": 20,
    "Otras_Deudas": 0
  }'
```

Request URL

http://localhost:8000/predict

Server response

Code	Details
200	Response body <pre>{ "prediction": 0, "probability": 0.63908825824481442 }</pre> Response headers <pre>content-length: 50 content-type: application/json date: Tue, 18 Aug 2026 04:14:23 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "prediction": 0,
  "probability": 1
}
```

Anexo III: Predicción de cumplimiento de pago 3

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict/batch' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "instances": [
      {
        "Edad": 17,
        "Nivel_Educacional": "Med",
        "Años_Trabajando": 0,
        "Ingresos": 0,
        "Deuda_Comercial": 0,
        "Deuda_Credito": 0,
        "Otras_Deudas": 0
      },
      {
        "Edad": 18,
        "Nivel_Educacional": "Med",
        "Años_Trabajando": 1,
        "Ingresos": 0,
        "Deuda_Comercial": 0,
        "Deuda_Credito": 0,
        "Otras_Deudas": 0
      }
    ]
  }'
```

Request URL

http://localhost:8000/predict/batch

Server response

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "type": "json_invalid", "loc": ["body", 717], "msg": "JSON decode error", "input": {}, "ctx": { "error": "Extra data" } }] }</pre> Response headers <pre>content-length: 121 content-type: application/json date: Tue, 18 Aug 2026 04:26:56 GMT server: uvicorn</pre>

```
{
  "instances": [
    {
      "Edad": 19,
      "Nivel_Educacional": "Med",
      "Años_Trabajando": 1,
      "Ingresos": 0,
      "Deuda_Comercial": 0,
      "Deuda_Credito": 0,
      "Otras_Deudas": 0
    }
  ]
}

{
  "instances": [
    {
      "Edad": 101,
      "Nivel_Educacional": "Med",
      "Años_Trabajando": 88,
      "Ingresos": 0,
      "Deuda_Comercial": 0,
      "Deuda_Credito": 0,
      "Otras_Deudas": 0
    }
  ]
}
}
```

Request URL

http://localhost:8000/predict/batch

Server response

Responses		
Code	Description	Links
200	Successful Response	No links
Media type application/json		
Controls Accept header		
Example Value Schema		
{ "predictions": [{ "prediction": 0, "probability": 1 }] }		
422	Validation Error	No links
Media type application/json		
Example Value Schema		
{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] }		

Anexo IV: CI/CD

alvarofj / Tarea_MLOps

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

← CI - Default Prediction API

ci: use supported Python test matrix #5

Summary

All jobs

- Lint (Ruff)
- Tests (Python 3.12)
- Tests (Python 3.13)
- Entrenar y validar modelo
- Build Docker y smoke test

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

alvarofj pushed • 866246 • main

Status: Success

Total duration: 2m 54s

Artifacts: 2

ci.yml on push

Lint (Ruff) 29s

Metric: test

2 jobs completed

Show all jobs

Entrenar y validar modelo 32s

Build Docker y smoke test 55s